

Reconstruction de surface

Reconstruction de Poisson en 2D et Cholesky creux sur grille

Objectif du devoir

L'objectif est la **reconstruction de surface** à partir d'un nuage de points, étudiée ici en 2D – la surface à reconstruire est une courbe. La technique de référence est la **reconstruction de Poisson** de Kazhdan et al. [1], adaptée à une grille cartésienne régulière. On estime une direction normale en chaque point par ajustement d'une droite moyenne, puis on oriente les normales de manière cohérente. Le champ obtenu par étalement gaussien permet de construire le second membre d'une équation de Poisson discrétisée par différences finies. Le solveur principal est un Cholesky creux, dont on étudiera le remplissage et l'influence de la numérotation des nœuds. La recherche spatiale, le parcours en largeur et la visualisation complètent cette chaîne de reconstruction.

1 Cahier des charges

La donnée est un nuage de N points $p_i = (x_i, y_i) \in \mathbb{R}^2$. Ce nuage de points est supposé provenir d'une courbe fermée ou ouverte, éventuellement bruitée et échantillonnée de manière non uniforme. On cherche à reconstruire la courbe à partir du nuage de points. Une application typique (en dimension 3) est la reconstruction d'une surface à partir d'un nuage de points 3D obtenu par un scanner.

On vous demande d'implémenter une reconstruction de Poisson en langage C. La signature de la fonction principale est

```
int reconstructionPoisson(int N, const double points[][2],
                          int k, double sigma,
                          double xmin, double ymin, double h,
                          int nx, int ny,
                          double *f, double *niveau);
```

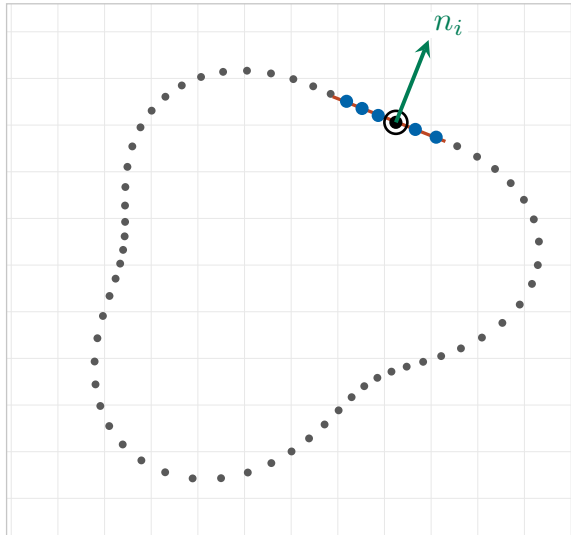
Les différents paramètres de cette fonction seront expliqués dans la suite du devoir.

Le projet se réalise par **groupes de quatre étudiants**. Le travail est assez long, mais se prête facilement à une répartition entre les membres du groupe : recherche des voisins, calcul et orientation des normales, étalement sur la grille, résolution par Cholesky creux et visualisation. Convenez dès le départ des interfaces et des structures de données communes, et prévoyez du temps pour assembler et tester l'ensemble.

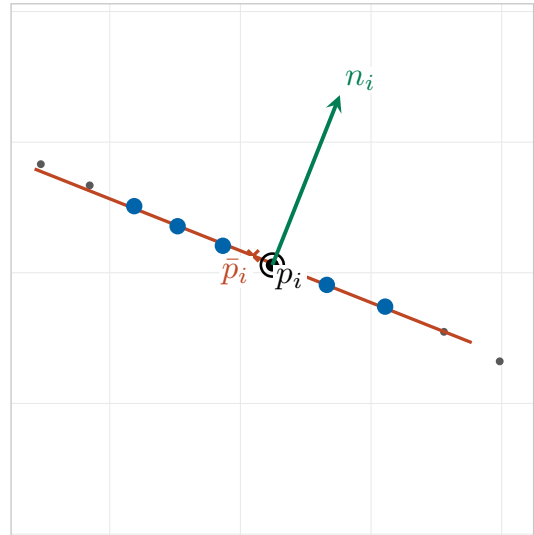
Vous présenterez votre projet par groupe, en répondant à un maximum de questions soulevées dans la suite du document.

2 Principe de la reconstruction de Poisson

Nuage issu d'une courbe fermée



Voisinage agrandi



• cinq voisins les plus proches

— droite moyenne

→ normale

FIGURE 1 – Un point p_i (entouré d'un cercle) et ses cinq plus proches voisins (bleus) dans un nuage de points issu d'une courbe fermée. La droite moyenne est ajustée sur ces six points et passe par leur barycentre $\bar{p}_i$. La flèche indique une normale unitaire à cette droite, placée en p_i ; son signe est choisi pour l'illustration et reste à déterminer par l'étape d'orientation. La grille et les proportions sont conservées dans l'agrandissement; la longueur de la flèche est adaptée à la lisibilité.

Avant de détailler les algorithmes, supposons que chaque point p_i du nuage soit muni d'une normale unitaire orientée n_i (Figure 1). On souhaite reconstruire une courbe à partir de ces points et de ces directions, sans connaître l'ordre des points le long de la courbe.

2.1 Construire une représentation implicite de la courbe

On cherche une **représentation implicite** de la courbe : une fonction $f(x, y)$ telle que la courbe cherchée soit décrite par

$$f(x, y) = \text{constante}.$$

Autrement dit, la courbe sera une **ligne de niveau** de f . Pour construire cette fonction numériquement, on superpose au nuage une grille cartésienne, c'est-à-dire un maillage du rectangle qui le contient. Les inconnues sont les valeurs $f_{r,s}$ de la fonction aux nœuds de cette grille. Une fois ces valeurs calculées, on choisit un niveau intermédiaire et on trace la ligne correspondante entre les nœuds. On conserve la notation f pour cette fonction dans tout le devoir; le niveau choisi est noté τ .

2.2 Pourquoi une équation de Poisson ?

Pour une courbe fermée, on souhaite idéalement obtenir une fonction presque constante à l'intérieur, presque constante à l'extérieur, avec deux valeurs différentes et une transition rapide au voisinage du nuage. Une ligne de niveau située entre ces deux valeurs suit alors la courbe à reconstruire.

Les normales orientées indiquent la direction dans laquelle cette transition doit se produire. On étale chaque normale avec une gaussienne et on additionne les contributions pour former un champ

$\mathbf{n}(x, y)$. Ce champ est fort près du nuage et faible loin de celui-ci. On cherche une fonction dont le gradient approche ce champ : $\nabla f \simeq \mathbf{n}$. Cela favorise une variation rapide à travers le nuage et de faibles variations de part et d'autre.

Voyons comment passer de cette relation vectorielle à une équation scalaire pour f . Si l'on pouvait imposer exactement $\nabla f = \mathbf{n}$, on aurait, composante par composante,

$$\frac{\partial f}{\partial x} = n^x, \quad \frac{\partial f}{\partial y} = n^y.$$

En dérivant la première égalité par rapport à x , la seconde par rapport à y , puis en les additionnant, on obtient

$$\underbrace{\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}}_{\Delta f} = \underbrace{\frac{\partial n^x}{\partial x} + \frac{\partial n^y}{\partial y}}_{\operatorname{div} \mathbf{n}}.$$

Cette opération consiste à prendre la **divergence** des deux membres. L'identité $\operatorname{div}(\nabla f) = \Delta f$ donne donc

$$\boxed{\Delta f = \operatorname{div} \mathbf{n} = g(x, y).}$$

Le champ $\mathbf{n}$ étant connu après l'étalement des normales, sa divergence fournit le second membre ; les valeurs de f sur la grille sont les inconnues du problème de Poisson.

Pour en comprendre l'intuition, considérons une traversée locale du nuage suivant la normale. La composante normale du champ forme une bosse : elle augmente à l'approche du nuage, puis diminue après l'avoir traversé. Sa dérivée contribue donc à un second membre g de signes opposés de part et d'autre. Dans ce modèle à une dimension, l'équation $f'' = g$ redonne une dérivée f' en forme de bosse, puis une fonction f qui passe progressivement d'un plateau à l'autre. Inverser toutes les normales inverse le sens de cette transition.

Cette image décrit le comportement recherché lorsque les normales sont cohérentes et le contour bien échantillonné. Une forte variation de g quelconque ne suffit pas à garantir deux plateaux : sa construction à partir des normales et les conditions aux limites sont essentielles.

2.3 Conditions aux limites sur le bord extérieur

On place le nuage de points dans un rectangle Ω dont le bord extérieur est suffisamment éloigné de la courbe à reconstruire. Sur ce bord, on impose une condition de Dirichlet homogène :

$$f(x, y) = 0 \quad \text{pour } (x, y) \in \partial\Omega.$$

Les valeurs de f sont donc fixées à zéro aux nœuds du bord de la grille. Cette condition fixe la référence de la fonction et assure l'unicité de la solution du problème de Poisson. La marge entre le nuage et le bord extérieur doit être suffisante pour limiter l'influence de cette condition sur la courbe reconstruite.

2.4 Recherche de la ligne de niveau

La courbe reconstruite est une ligne de niveau de la solution :

$$\Gamma = \{(x, y) \in \Omega : f(x, y) = \tau\}.$$

Le niveau τ n'est pas nécessairement zéro : la condition au bord du rectangle n'impose pas la valeur de f sur le nuage. On choisira τ comme une moyenne pondérée des valeurs de f interpolées aux points p_i , puis on extraira la ligne correspondante sur la grille. La fonction f n'est pas une distance signée et le contour obtenu ne passe pas nécessairement exactement par tous les points bruités.

3 Calcul de normales cohérentes

On doit **attribuer une normale à chaque point du nuage**. Pour cela, on commence par chercher les k plus proches voisins de chaque point p_i . On ajuste ensuite un plan moyen à ce voisinage en dimension trois, ou une droite moyenne dans le cas bidimensionnel de ce devoir. La perpendiculaire à cet ajustement fournit une **direction normale**.

À ce stade, on connaît une direction, mais pas encore un sens : les deux vecteurs unitaires n_i et $-n_i$ conviennent tout autant. L'ajustement est effectué indépendamment en chaque point ; il peut donc produire des normales dont le sens s'inverse d'un point au suivant.

Il faut ensuite **orienter les normales de manière cohérente**. Sur une même portion de courbe suffisamment bien échantillonnée, les normales doivent évoluer progressivement, sans inversions arbitraires.

3.1 Recherche spatiale

3.1.1 Trouver les k points les plus proches

Pour estimer la tangente et la normale près d'un point, il faut d'abord identifier un voisinage local dans le nuage non ordonné. Pour un point requête $q = (x, y)$, la recherche consiste à sélectionner les k points qui ont les plus petites distances euclidiennes à q . On peut comparer les distances au carré, sans calculer de racines carrées :

$$d^2(q, p_j) = (x - x_j)^2 + (y - y_j)^2.$$

Dans ce devoir, chaque point p_i devient successivement le point requête. On note $\mathcal{N}_k(i)$ l'ensemble des indices de ses k voisins, avec $1 \leq k < N$, en excluant i lui-même. Des points proches dans le fichier ne sont pas nécessairement proches dans le plan. De même, la proximité euclidienne ne garantit pas l'appartenance à la même branche de la courbe.

3.1.2 Pourquoi une structure de recherche spatiale ?

La méthode exhaustive calcule les $N - 1$ distances pour chaque requête, puis retient les k plus petites. Un tri complet coûte $\mathcal{O}(N \log N)$ par requête ; une sélection partielle évite de tout trier. Cependant, calculer toutes les distances pour les N points représente déjà $\mathcal{O}(N^2)$ opérations. Cette méthode est une bonne référence de validation sur de petits nuages, mais devient coûteuse lorsque N augmente.

Une structure spatiale est construite une seule fois et réutilisée pour les requêtes. Elle permet d'écarter des régions entières qui ne peuvent pas contenir de meilleur voisin. Plusieurs techniques sont possibles :

- **Arbre k -d (kd-tree)**. Le nuage est partagé récursivement suivant les coordonnées x ou y . La recherche visite d'abord les régions proches de la requête et ignore celles dont la distance minimale dépasse celle du plus éloigné des k candidats courants. Le symbole k du nom de l'arbre désigne ici la dimension, et non le nombre de voisins.
- **Grille de cellules (uniform grid)**. Les points sont répartis dans des cellules du plan. On explore la cellule de la requête, puis les cellules voisines. Pour une recherche exacte, on poursuit tant qu'une cellule non visitée peut contenir un point plus proche que le dernier candidat retenu. La taille des cellules influe sur l'efficacité.
- **Arbre quaternaire (quadtree)**. Chaque région carrée est subdivisée en quatre lorsque son occupation devient trop importante. Ce découpage adapte la résolution aux variations de densité du nuage.

Ces structures réduisent souvent fortement le travail en 2D, mais leur coût dépend de la distribution des points et des paramètres choisis. Une requête peut encore nécessiter un parcours important dans un cas défavorable ; on ne supposera pas un coût logarithmique garanti.

3.1.3 Calcul des adjacences avec FLANN

Construisez un arbre de recherche avec **FLANN** [2], puis calculez les k plus proches voisins de chaque point p_i pour remplir les listes d'adjacence suivantes. La fonction est à compléter en utilisant l'interface C de FLANN.

```
typedef struct {
    int N;           /* Nombre de points du nuage. */
    int k;           /* Nombre de voisins par point. */
    int *voisins;     /* Tableau de N*k indices. */
} Adjacences;

void calculerAdjacences(const double points[][2],
                       Adjacences *adj)
{
    /* À compléter avec FLANN. */
}
```

La fonction C appelante initialise `adj->N` et `adj->k`, avec $1 \leq k < N$, et alloue le tableau `adj->voisins` de Nk entiers. La fonction y écrit les résultats : `adj->voisins[i*k + j]` contient l'indice du voisin numéro j du point i , avec des indices C commençant à zéro. Le point i est exclu de sa propre liste par son indice, même si d'autres points ont les mêmes coordonnées. Ces listes ne sont pas nécessairement réciproques ; elles serviront ensuite à construire le graphe non orienté de la section sur l'orientation.

3.2 Droite moyenne et normales locales

À partir des coordonnées des points et des adjacences calculées à la section précédente, calculez, pour chaque point p_i , la droite moyenne ajustée au point lui-même et à ses k plus proches voisins, au sens des moindres carrés des distances orthogonales. Déduisez-en une direction normale unitaire $\tilde{n}_i$; son sens sera fixé à l'étape suivante.

Pour le voisinage $V_i = \{i\} \cup \mathcal{N}_k(i)$, le barycentre et la matrice de covariance sont

$$\bar{p}_i = \frac{1}{|V_i|} \sum_{j \in V_i} p_j, \quad C_i = \frac{1}{|V_i|} \sum_{j \in V_i} (p_j - \bar{p}_i)(p_j - \bar{p}_i)^T.$$

La normale unitaire $\tilde{n}_i$ est un vecteur propre de C_i associé à sa plus petite valeur propre. La droite moyenne s'écrit

$$D_i = \{q \in \mathbb{R}^2 : \tilde{n}_i^T (q - \bar{p}_i) = 0\}.$$

```
void calculerDroitesMoyennes(const double points[][2],
                             const Adjacences *adj,
                             double barycentres[][2],
                             double normales[][2]);
```

La fonction C appelante alloue les tableaux `barycentres` et `normales`, chacun de N paires de réels. Pour chaque point p_i , la fonction y écrit le barycentre du voisinage et une normale unitaire à la droite moyenne. Le barycentre et la normale définissent cette droite ; le signe de la normale reste arbitraire à ce stade.

3.3 Graphe de connectivité et orientation par parcours en largeur (breadth-first search)

Les adjacences permettent de construire un graphe dont les sommets sont les points du nuage. On le rend non orienté : les points i et j sont reliés si $j \in \mathcal{N}_k(i)$ ou $i \in \mathcal{N}_k(j)$. Il faut donc tenir compte des voisins déclarés dans les deux sens, car les listes calculées avec FLANN ne sont pas nécessairement réciproques.

Le **parcours en largeur** utilise une file : le premier point ajouté est le premier traité. On choisit un point de départ, on conserve le sens de sa normale, on le marque comme découvert et on l'ajoute à la file. On retire ensuite un point i de la file et on examine ses voisins. Pour chaque voisin j non encore découvert, on choisit le sens de sa normale pour qu'elle forme un angle aigu ou droit avec celle de i :

$$n_j = \begin{cases} \tilde{n}_j, & n_i \cdot \tilde{n}_j \geq 0, \\ -\tilde{n}_j, & n_i \cdot \tilde{n}_j < 0. \end{cases}$$

On marque immédiatement j comme découvert et on l'ajoute à la file. On continue jusqu'à ce que la file soit vide. S'il reste des points non découverts, on recommence depuis l'un d'eux : chaque composante connexe est ainsi parcourue. Une normale déjà orientée n'est pas modifiée lors d'une rencontre ultérieure.

Implémentez cette propagation à partir des adjacences et des normales locales, avec le prototype suivant :

```
void orienterNormales(const Adjacences *adj,
                     double normales[][2]);
```

Le tableau **normales**, calculé à l'étape précédente, contient N paires de réels. La fonction modifie les normales en place, uniquement en changeant leur signe. Elle construit le graphe non orienté à partir de **adj** et gère la file et les marques de découverte.

Difficultés possibles. Une arête peut relier deux branches proches mais distinctes et propager un mauvais sens. Un produit scalaire presque nul rend le choix du signe fragile. De plus, les arêtes hors de l'arbre de parcours peuvent révéler des contradictions entre les orientations déjà fixées. Enfin, chaque composante conserve un signe global arbitraire : ce parcours ne détermine pas à lui seul l'intérieur et l'extérieur, notamment pour des contours emboîtés. Ces difficultés doivent guider l'interprétation des résultats.

3.4 Étalement gaussien des normales (gaussian splatting)

L'**étalement gaussien** consiste à répartir l'influence de chaque point du nuage dans son voisinage. Imaginez que chaque point dépose une petite tache sur la grille : son influence est maximale au centre et diminue progressivement lorsqu'on s'en éloigne, selon une gaussienne. Ici, chaque tache porte la normale du point. En chaque nœud de la grille, on additionne les normales des points voisins, pondérées par leur influence. On obtient ainsi un champ de vecteurs à partir de normales connues seulement aux points du nuage. Un paramètre σ , qui est une longueur, règle la largeur de ces taches.

La gaussienne est normalisée pour que son intégrale sur tout le plan soit égale à un. Changer sa largeur répartit donc la même contribution totale sur une zone plus ou moins grande, sans en changer le poids global. À poids a_i égaux, chaque normale apporte ainsi la même contribution globale, quelle que soit la valeur de σ . Cette propriété, ainsi que la décroissance régulière de la gaussienne, en fait un choix naturel pour l'étalement. Ce n'est toutefois pas le seul choix : d'autres

noyaux normalisés à une intégrale de un peuvent être utilisés. Sur une grille finie, cette conservation reste approchée à cause de la discrétisation et de la partie du noyau située hors du domaine.

On représente la grille cartésienne par la structure suivante :

```
typedef struct {
    int nx, ny;           /* Dimensions, bord compris. */
    double xmin, ymin;    /* Origine de la grille. */
    double h;             /* Pas identique en x et y. */
    double *g;            /* Divergence : nx*ny valeurs. */
    double *f;            /* Solution : nx*ny valeurs. */
} Grille;
```

Les nœuds ont pour coordonnées

$$q_{r,s} = (x_{\min} + rh, y_{\min} + sh), \quad 0 \leq r < n_x, \quad 0 \leq s < n_y.$$

La fonction C appelante initialise ces paramètres, avec $h > 0$ et $n_x, n_y \geq 3$, et alloue les tableaux $\mathbf{g}$ et $\mathbf{f}$, chacun de $n_x n_y$ réels. Le rectangle doit contenir le nuage avec une marge. La valeur au nœud (r, s) est stockée à l'indice $\mathbf{r} + \mathbf{nx} * \mathbf{s}$ dans les deux tableaux. Le tableau $\mathbf{f}$ recevra la solution de Poisson, avec des valeurs nulles au bord.

Chaque normale orientée n_i est étalée autour du point p_i par une gaussienne de largeur $\sigma > 0$:

$$G_\sigma(d) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{d^2}{2\sigma^2}\right), \quad \mathbf{n}(q) = \sum_{i=1}^N a_i n_i G_\sigma(\|q - p_i\|).$$

Les poids positifs a_i peuvent tous être fixés à 1 dans une première version. Des poids adaptés à l'espacement local permettent de tenir compte d'un échantillonnage non uniforme.

En dérivant les gaussiennes, on calcule directement la divergence en chaque nœud de grille à partir des normales du nuage :

$$g(q) = \operatorname{div} \mathbf{n}(q) = -\frac{1}{\sigma^2} \sum_{i=1}^N a_i [n_i \cdot (q - p_i)] G_\sigma(\|q - p_i\|).$$

Implémentez ce calcul avec le prototype suivant :

```
void calculerDivergence(int N, const double points[][2],
                        const double normales[][2],
                        const double poids[], double sigma,
                        Grille *grille);
```

Les tableaux `points`, `normales` et `poids` contiennent respectivement les N points, leurs normales orientées et les poids a_i . La fonction remplit `grille->g` à tous les nœuds, bord compris ; seules les valeurs intérieures serviront au second membre du système de Poisson.

Le choix de σ est essentiel. Il représente la largeur sur laquelle chaque point influence le champ. Testez plusieurs valeurs et interprétez leur effet sur la reconstruction : un σ trop petit par rapport au pas de grille ou à l'espacement des points donne un champ mal résolu ou discontinu en apparence ; un σ trop grand lisse les détails et peut fusionner des branches proches.

4 Équation de Poisson et différences finies

On résout $\Delta f = g$, avec $g = \operatorname{div} \mathbf{n}$, sur une grille cartésienne de pas h . On utilise le **schéma à cinq points** vu au cours. En notant $f_{r,s} \simeq f(q_{r,s})$, on écrit à chaque nœud intérieur :

$$\frac{f_{r+1,s} + f_{r-1,s} + f_{r,s+1} + f_{r,s-1} - 4f_{r,s}}{h^2} = g_{r,s}.$$

Le second membre $g_{r,s} = g(q_{r,s})$ est fourni par la fonction `calculerDivergence` de la section précédente. Les valeurs de f sur le bord extérieur sont fixées à zéro. Seules les valeurs aux nœuds intérieurs sont inconnues ; lorsqu'un voisin est sur le bord, sa valeur connue intervient dans le schéma. Numérotez les inconnues intérieures pour former le système linéaire à résoudre.

Complétez la fonction suivante pour résoudre le système de Poisson :

```
void resoudrePoisson(Grille *grille)
{
    /* À compléter : résolution par Cholesky creux. */
}
```

La fonction utilise le second membre contenu dans `grille->g` et écrit la solution dans `grille->f`, déjà alloué par la fonction C appelante. Les valeurs de `f` au bord sont fixées à zéro. La factorisation de Cholesky creuse est détaillée dans la section suivante.

5 Cholesky creux et numérotation des nœuds

Écrivez le système sous la forme $AF = b$, où F contient les valeurs inconnues de f , $A = -\Delta_h$ et b contient les valeurs $-g_{r,s}$ aux nœuds intérieurs. Le nombre d'inconnues est $M = (n_x - 2)(n_y - 2)$. Justifiez que A est symétrique définie positive avec les conditions de Dirichlet choisies. **On demande une factorisation de Cholesky creuse**, ainsi que les substitutions triangulaires associées, implémentées en C. Le stockage et les calculs doivent exploiter les coefficients utiles de la matrice et de son facteur.

5.1 Remplissage et stockage creux

La matrice du schéma à cinq points est creuse, mais son facteur de Cholesky possède généralement de nouveaux coefficients non nuls (fill-in) : c'est le **remplissage**. Dans le graphe de la matrice, l'élimination d'un nœud peut créer des connexions entre ses voisins non encore éliminés. La structure de L dépend donc de l'ordre dans lequel on élimine les inconnues.

Le format **CSC** (*Compressed Sparse Column*) range les coefficients stockés **colonne par colonne**, dans trois tableaux. Le tableau `valeurs` contient les coefficients, et `lignes` contient leurs indices de ligne. Le tableau `debutColonne` indique où commence chaque colonne dans les deux premiers tableaux. Une position absente du stockage représente un zéro.

```
#include <stddef.h>

typedef struct {
    int n; /* Matrice carree de taille n. */
    size_t nnz; /* Nombre de positions stockees. */
    double *valeurs; /* nnz coefficients. */
    int *lignes; /* nnz indices de ligne. */
    size_t *debutColonne; /* n+1 positions dans les tableaux. */
} MatriceCSC;
```

Les indices commencent à zéro. Les coefficients de la colonne j occupent les positions allant de `debutColonne[j]` (inclusive) à `debutColonne[j+1]` (exclue). À la position p , on trouve donc le coefficient de ligne `lignes[p]` et de colonne j , dont la valeur est `valeurs[p]`. On a `debutColonne[0] = 0` et `debutColonne[n] = nnz`; la dernière entrée marque la fin du stockage, et non le début d'une colonne supplémentaire.

Par exemple, le triangle inférieur de la matrice symétrique

$$A = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 5 & 2 \\ 0 & 2 & 6 \end{pmatrix}$$

est stocké ainsi :

```
/* n = 3, nnz = 5 */
double valeurs[]      = {4, 1, 5, 2, 6};
int lignes[]          = {0, 1, 1, 2, 2};
size_t debutColonne[] = {0, 2, 4, 5};
```

La première colonne contient les valeurs 4 et 1, la deuxième 5 et 2, la troisième 6. Pour A , le triangle supérieur se déduit par symétrie. Pour L , il est nul. Dans les deux cas, les indices de ligne sont triés et sans doublons dans chaque colonne; la diagonale est stockée en premier. La même structure `MatriceCSC` sert pour A et L , mais leurs tableaux sont distincts puisque le facteur contient les positions créées par le remplissage.

À partir de la structure `Grille`, construisez la matrice $A = -\Delta_h$ du schéma à cinq points dans une `MatriceCSC`. Seuls les nœuds intérieurs sont des inconnues, soit $M = (n_x - 2)(n_y - 2)$. Utilisez d'abord une numérotation ligne par ligne et stockez uniquement le triangle inférieur : les coefficients diagonaux valent $4/h^2$ et les coefficients reliant deux nœuds intérieurs voisins valent $-1/h^2$. Les nœuds du bord, où $f = 0$, ne correspondent à aucune ligne ni colonne de la matrice.

```
void construireMatricePoisson(const Grille *grille,
                             const int numero[],
                             MatriceCSC *A);
```

La fonction `C` appelante fournit une structure `MatriceCSC` vide et le tableau `numero` de M indices. Pour la numérotation classique, on prend `numero[i] = i`; son utilisation lors de l'assemblage est précisée dans la section suivante. La fonction initialise `A->n` et `A->nnz`, alloue les trois tableaux à la taille nécessaire et les remplit. Les indices de ligne doivent être croissants dans chaque colonne. La fonction `C` appelante libérera ensuite les tableaux alloués.

Effectuez ensuite une **factorisation de Cholesky symbolique**. À partir de la structure de A , déterminez exactement les positions (i, j) à stocker dans le triangle inférieur du facteur L , y compris les nouveaux coefficients créés par le remplissage. Cette étape calcule la structure du facteur, et non les valeurs numériques de ses coefficients. Elle permet d'allouer les tableaux CSC de L à la taille nécessaire.

Principe : simuler les connexions créées par l'élimination. Considérez le graphe non orienté de A : une inconnue correspond à un sommet, et chaque coefficient hors diagonale stocké A_{ij} à une arête entre i et j . Le triangle inférieur suffit pour reconstruire ce graphe; une arête se parcourt dans les deux sens. Les coefficients diagonaux ne créent pas d'arêtes.

Lorsqu'on élimine l'inconnue j , les coefficients entre ses voisins encore présents sont mis à jour. Pour deux de ces voisins u et v , la mise à jour numérique aurait la forme

$$a_{uv}^{\text{nouveau}} = a_{uv}^{\text{courant}} - \ell_{uj}\ell_{vj}.$$

Même si a_{uv}^{courant} était nul, cette opération peut créer un coefficient. L'analyse symbolique retient donc la position (u, v) , sans effectuer le calcul : **les voisins restants de j doivent tous être reliés entre eux avant de poursuivre l'élimination.**

Algorithme à suivre. Travaillez sur une copie de la structure du graphe, avec des ensembles de voisins qui permettent l'ajout d'une arête sans doublon. Pour $j = 0, \dots, M - 1$, dans l'ordre de numérotation choisi :

1. Relevez l'ensemble S_j des voisins de j non encore éliminés, c'est-à-dire ceux dont l'indice est supérieur à j . Cet ensemble comprend les voisins initiaux *et* ceux ajoutés par les étapes précédentes.
2. Enregistrez la structure de la colonne j de L : la ligne diagonale j , suivie des indices de S_j triés par ordre croissant. Cette colonne contient donc $1 + |S_j|$ positions.
3. Pour chaque paire de sommets distincts de S_j , ajoutez une arête si elle n'existe pas déjà. Ces nouvelles arêtes représentent le remplissage qui influencera les colonnes suivantes.
4. Retirez j du graphe actif, ou marquez-le comme éliminé pour qu'il ne soit plus pris en compte.

L'étape d'ajout des arêtes est essentielle : conserver uniquement les voisins de la matrice initiale donnerait une structure incomplète du facteur. Une liste dynamique par sommet, avec suppression des doublons, permet d'implémenter cette méthode sans matrice d'adjacence dense.

Exemple sur quatre inconnues. Prenez une petite grille dont les arêtes sont $(0, 1)$, $(0, 2)$, $(1, 3)$ et $(2, 3)$. Au départ, $S_0 = \{1, 2\}$. La colonne 0 de L contient donc les lignes 0, 1, 2. L'élimination de 0 ajoute l'arête $(1, 2)$: c'est une position de remplissage. À l'étape suivante, $S_1 = \{2, 3\}$, et non simplement $\{3\}$. Les listes de lignes des quatre colonnes de L sont finalement

$$(0, 1, 2), \quad (1, 2, 3), \quad (2, 3), \quad (3).$$

Elles donnent 9 positions dans L , contre 8 dans le triangle inférieur de A , diagonale comprise.

Passage au stockage CSC. Conservez les listes de lignes des colonnes pendant le parcours. Leurs tailles donnent les positions de début par sommes cumulées :

$$\text{debutColonne}[0] = 0, \quad \text{debutColonne}[j + 1] = \text{debutColonne}[j] + 1 + |S_j|.$$

La dernière position donne **nnz**. Allouez alors le tableau **lignes** et recopiez-y les listes, colonne par colonne, puis allouez **valeurs** à la même taille. Les valeurs seront calculées lors de la factorisation numérique. Pour l'exemple précédent :

```
/* Structure du facteur L : n = 4, nnz = 9. */
size_t debutColonne[] = {0, 3, 6, 8, 9};
int lignes[]          = {0, 1, 2, 1, 2, 3, 2, 3, 3};
```

On détermine ainsi la structure exacte prédite par l'élimination. D'éventuelles annulations numériques ne sont pas recherchées : une position structurellement nécessaire reste réservée, même si sa valeur calculée se révèle nulle. Libérez les listes temporaires après leur conversion au format CSC.

```
void choleskySymbolique(const MatriceCSC *A,
                       MatriceCSC *L);
```

La fonction `C` appelante fournit une structure `L` vide. La fonction initialise `L->n` et `L->nnz`, alloue et remplit `L->debutColonne` et `L->lignes`, puis alloue `L->valeurs` pour la factorisation numérique ultérieure. Les indices de ligne sont croissants dans chaque colonne et la diagonale est incluse. La structure doit contenir toutes les positions nécessaires, sans réserver une bande complète ou une matrice dense.

Cette analyse dépend de la numérotation des inconnues. Appliquez-la à la matrice assemblée avec la numérotation choisie, comme expliqué dans la section suivante.

Factorisation numérique de Cholesky. Une fois la structure de L calculée, remplissez son tableau `valeurs` pour obtenir $A = LL^T$. Pour chaque colonne $j = 0, \dots, M - 1$, calculez d'abord le coefficient diagonal :

$$\ell_{jj} = \sqrt{a_{jj} - \sum_{k=0}^{j-1} \ell_{jk}^2},$$

puis les coefficients sous la diagonale dont les positions ont été prévues par l'analyse symbolique :

$$\ell_{ij} = \frac{1}{\ell_{jj}} \left(a_{ij} - \sum_{k=0}^{j-1} \ell_{ik} \ell_{jk} \right), \quad i > j.$$

Une position absente de A correspond à $a_{ij} = 0$, mais peut produire un coefficient non nul de L . Exploitez les listes d'indices du stockage creux pour ne parcourir que les termes utiles des sommes, sans construire de matrice dense. Les tableaux `lignes` et `debutColonne` de L restent ceux de l'analyse symbolique.

```
int choleskyNumerique(const MatriceCSC *A,
                     MatriceCSC *L);
```

La fonction reçoit A et le facteur L préparé par `choleskySymbolique`. Elle calcule les valeurs du facteur et renvoie zéro en cas de succès, une valeur non nulle si un pivot nul ou négatif est rencontré avant la racine carrée. Pour la matrice de Poisson considérée, les pivots sont strictement positifs en arithmétique exacte. Les analyses symbolique et numérique portent sur la même matrice, assemblée directement avec la numérotation choisie.

Substitutions avant et arrière. Une fois $A = LL^T$ factorisée, résolvez $AF = b$ en deux étapes. La **substitution avant** résout $Ly = b$, dans l'ordre croissant $i = 0, \dots, M - 1$:

$$y_i = \frac{b_i - \sum_{j < i} \ell_{ij} y_j}{\ell_{ii}}.$$

La **substitution arrière** résout ensuite $L^T F = y$, dans l'ordre décroissant $i = M - 1, \dots, 0$:

$$F_i = \frac{y_i - \sum_{j > i} \ell_{ji} F_j}{\ell_{ii}}.$$

Exploitez directement le stockage CSC de L . Pour la substitution avant, une fois y_i calculé, parcourez la colonne i pour soustraire sa contribution aux seconds membres des lignes suivantes. Pour la substitution arrière, la colonne i contient les coefficients ℓ_{ji} nécessaires à la somme. Il n'est pas nécessaire de construire explicitement L^T .

```
void substitutionAvant(const MatriceCSC *L,
                     const double b[], double y[]);
void substitutionArriere(const MatriceCSC *L,
                       const double y[], double F[]);
```

La fonction `C` appelante alloue les tableaux de sortie de M réels. Les vecteurs utilisent la même numérotation que la matrice assemblée. La solution F est ensuite replacée dans `grille->f` en tenant compte du tableau `numero`.

Validation par une solution fabriquée. Testez d’abord le solveur indépendamment du nuage et des normales. Sur $\Omega = [0, 1]^2$, choisissez la solution connue

$$f_\star(x, y) = \sin(\pi x) \sin(\pi y).$$

Elle est nulle au bord. Pour qu’elle satisfasse $\Delta f_\star = g_\star$, il faut choisir le second membre

$$g_\star(x, y) = -2\pi^2 \sin(\pi x) \sin(\pi y).$$

Remplissez donc `grille->g` avec les valeurs de $g_\star$, sans appeler `calculerDivergence`. Puisque la matrice assemblée représente $-\Delta_h$, le second membre du système $AF = b$ contient **les valeurs de** $-g_\star$, placées selon `numero`. Comparez la solution obtenue à $f_\star$ aux nœuds intérieurs et mesurez

$$e_h = \max_{r,s \text{ intérieurs}} |f_{r,s} - f_\star(q_{r,s})|.$$

En divisant h par deux, l’erreur doit être approximativement divisée par quatre, tant que les erreurs d’arrondi restent négligeables. Cette différence est une erreur de discrétisation : les valeurs de la solution continue ne satisfont pas exactement le système discret.

Pour isoler la factorisation et les substitutions de cette erreur, on peut aussi former le vecteur $F_\star$ des valeurs connues et choisir $b = AF_\star$, calculé avec la matrice originale ou son schéma à cinq points. Le solveur doit alors retrouver $F_\star$ à l’erreur numérique près, sans erreur de discrétisation dans ce test algébrique.

Validation et performances. Vérifiez le facteur sur une petite grille et mesurez le résidu relatif $\|AF - b\|_2 / \|b\|_2$ dans la numérotation utilisée pour l’assemblage ; si $b = 0$, rapportez un résidu absolu. Utilisez la matrice originale pour ce calcul, et non le produit des facteurs calculés.

Mesurez séparément l’assemblage, le calcul de la permutation, l’analyse symbolique, la factorisation numérique et les substitutions. Reliez les coûts observés au remplissage et à la structure de L . Comparez éventuellement à un Cholesky dense sur les petites tailles seulement. À grille, domaine et conditions au bord fixés, réutilisez la permutation et le facteur pour plusieurs seconds membres, par exemple en modifiant l’orientation des normales ou σ . Distinguez le coût de la première reconstruction de celui des suivantes.

Comparaison facultative. Essayez un autre solveur, par exemple un gradient conjugué éventuellement préconditionné. Une bibliothèque est autorisée pour cette comparaison. À précision comparable, comparez le temps total, la mémoire et la qualité de la reconstruction à votre Cholesky creux. Incluez le coût de préparation, le critère d’arrêt et le nombre d’itérations.

5.2 Quelle numérotation des nœuds choisir ?

Une numérotation classique, ligne par ligne, permet de construire la matrice et de réaliser la factorisation de Cholesky. Elle fonctionne, mais n’est généralement pas optimale pour limiter le remplissage et le coût de la factorisation. Changer l’ordre des inconnues peut réduire le nombre de coefficients à stocker et à calculer.

Deux stratégies peuvent être utilisées :

- **Cuthill–McKee inversé (RCMK).** Cette méthode, expliquée au cours, effectue un parcours en largeur du graphe en visitant les voisins par degré croissant, puis inverse l’ordre obtenu. Elle vise à rapprocher les coefficients non nuls de la diagonale et à réduire la largeur de bande. Son effet sur le remplissage dépend de la matrice.

- **METIS : dissection emboîtée.** On sépare le graphe en sous-domaines, puis on élimine les inconnues à l'intérieur de chaque sous-domaine avant celles des séparateurs. Ce découpage est répété récursivement. La bibliothèque **METIS** [3] calcule une telle numérotation avec la fonction `METIS_NodeND`, utilisable depuis le C.

Vous pouvez utiliser une bibliothèque pour calculer la numérotation, sans réimplémenter ces méthodes. On calcule uniquement un tableau de permutation `numero` : `numero[i]` donne le numéro de l'inconnue associée au nœud intérieur i , initialement repéré dans l'ordre ligne par ligne. Ce tableau contient une fois chacun des entiers de 0 à $M - 1$.

La numérotation est prise en compte directement lors de l'assemblage de la matrice. Un coefficient reliant les nœuds i et j est placé aux indices `numero[i]` et `numero[j]`. Pour ne stocker que le triangle inférieur, on utilise le plus grand de ces deux indices comme ligne et le plus petit comme colonne, en insérant chaque liaison une seule fois. Les indices de ligne sont ensuite triés dans chaque colonne CSC.

On ne construit donc ni matrice de permutation ni produit de matrices pour renuméroter A . Le second membre est lui aussi assemblé directement dans cet ordre : `b[numero[i]]` reçoit la valeur $-g$ au nœud i . Les étapes symbolique et numérique s'appliquent à la matrice ainsi assemblée. Après les substitutions triangulaires, la valeur `F[numero[i]]` est remplacée au nœud i dans `grille->f`.

6 Extraction et représentation de la courbe

La solution f n'est pas une distance signée et son niveau sur la courbe n'est pas imposé. Le niveau recherché n'est donc pas nécessairement zéro. Évaluez f aux points du nuage par interpolation bilinéaire. En notant f_h cet interpolant, utilisez comme niveau de référence

$$\tau = \frac{\sum_i a_i f_h(p_i)}{\sum_i a_i}, \quad \Gamma_h = \{(x, y) \in \Omega : f_h(x, y) = \tau\}.$$

Extrayez cette ligne de niveau, par exemple par interpolation des intersections avec les arêtes des cellules ou par une routine de contour documentée. Précisez le traitement des cellules ambiguës et conservez les composantes parasites dans les figures de diagnostic.

1. Superposez les points et Γ_h avec un rapport d'aspect égal sur les deux axes. Affichez aussi une carte de f , le niveau retenu et un zoom sur les zones où la reconstruction est difficile.
2. Étudiez la dispersion des $f_h(p_i)$ autour de τ et la sensibilité au choix du niveau. Comparez l'effet d'une inversion de *toutes* les normales à celui d'une inversion sur une seule composante, en recalculant τ dans les deux cas.
3. Pour les cas synthétiques, mesurez une erreur géométrique dans les deux sens entre la courbe reconstruite et la courbe de référence (par exemple des distances point-segment sur des échantillons denses). Une distance des seuls points d'entrée à la reconstruction ne détecte pas toutes les composantes parasites. Rapportez également le nombre de composantes et la préservation ou la perte des trous.

La fonction `reconstructionPoisson` écrit le champ calculé dans le tableau `f`, alloué par la fonction C appelante avec $n_x n_y$ éléments : `f[r + nx*s]` contient $f_{r,s}$, bord nul inclus. Après une renumérotation pour Cholesky, restituez donc l'ordre initial de la grille. La fonction écrit τ dans `*niveau` et renvoie zéro en cas de succès, une valeur non nulle en cas d'échec. L'extraction et le tracé du contour utilisent ensuite ces sorties ; vous pouvez employer un outil de visualisation ou une bibliothèque de contour en l'identifiant.

7 Expériences, cas limites et travail à remettre

Les familles de cas suivantes guideront votre étude et la présentation par groupe. Pour les cas étudiés, fournissez une configuration reproductible et une interprétation des résultats, y compris lorsque la méthode échoue.

Cas étudié	Question à examiner
Cercle ou ellipse bien échantillonné	Validation de référence ; convergence lorsque le nuage et la grille sont raffinés.
Courbe fortement concave et branches proches	Arêtes incorrectes, propagation des signes et fusion par l'étalement gaussien.
Deux composantes séparées et deux contours emboîtés	Signes indépendants après parcours en largeur ; orientation relative nécessaire à la reconstruction d'un domaine avec trou.
Bruit et densité non uniforme	Fiabilité de l'ajustement local ; influence de k , des poids et de σ .
Lacune importante ou arc ouvert ; points aberrants	Fermeture artificielle, composantes parasites et absence d'un intérieur bien défini pour une courbe ouverte.

Faites varier séparément les paramètres pour identifier l'origine des effets observés. En particulier, distinguez le raffinement de la grille à nuage et σ fixés d'une amélioration de l'échantillonnage. Un petit résidu du système linéaire ne garantit pas une bonne reconstruction.

7.1 Exemple de programme principal

Le programme suivant montre les allocations et l'enchaînement des étapes sur un nuage lu dans un fichier texte passé en argument, par exemple `./reconstruction nuage.txt -N 61`. La première ligne contient le nombre de points `np`, puis chaque ligne contient deux coordonnées :

```
np
x y
x y
...
```

Ici, `np` désigne un entier et `x y` deux réels, à remplacer par leurs valeurs dans le fichier. L'option obligatoire `-N` fixe le nombre de nœuds par côté de la grille, bord compris (au moins trois) ; elle ne désigne pas le nombre de points du nuage. Les options `-k` et `-sigma` sont facultatives : leurs valeurs par défaut sont $k = 5$ et $\sigma = 1,5h$. Il faut $1 \leq k < np$ et $\sigma > 0$; σ est exprimé dans la même unité de longueur que les coordonnées. Par exemple :

```
./reconstruction nuage.txt -N 101 -sigma 0.05 -k 10
```

Le programme adapte la grille au nuage avec une marge de 10 % de sa plus grande étendue de chaque côté.

Le programme principal lit les points, alloue le tableau de sortie `f`, puis appelle `reconstructionPoisson` avec la signature du cahier des charges. Cette fonction réalise toutes les étapes de reconstruction et renvoie le champ et le niveau à tracer. Le type `Grille` est supposé déclaré comme précédemment. La fonction utilitaire `exporterResultats`, à écrire, exporte le nuage, la grille et le niveau ; elle renvoie zéro en cas de succès.

```
#include <errno.h>
```

```

#include <limits.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Ajouter ici la definition de Grille. */
int reconstructionPoisson(int N, const double points[][2],
                          int k, double sigma,
                          double xmin, double ymin, double h,
                          int nx, int ny,
                          double *f, double *niveau);
int exporterResultats(const char *fichier, int N,
                     const double points[][2],
                     const Grille *grille, double niveau);

int main(int argc, char *argv[])
{
    int k = 5, nGrille = 0;
    double sigma = 0.0; /* Zero indique la valeur par default. */
    const char *fichier = NULL;
    int N; /* Nombre de points lu dans le fichier. */
    for (int a = 1; a < argc; ++a) {
        const char *option = argv[a];
        if (strcmp(option, "-N") == 0 ||
            strcmp(option, "-k") == 0 ||
            strcmp(option, "-sigma") == 0) {
            if (++a == argc) goto usage;
            char *finValeur;
            errno = 0;
            if (strcmp(option, "-sigma") == 0) {
                sigma = strtod(argv[a], &finValeur);
                if (errno || finValeur == argv[a] || *finValeur ||
                    !isfinite(sigma) || sigma <= 0.0) goto usage;
            } else {
                long valeur = strtol(argv[a], &finValeur, 10);
                if (errno || finValeur == argv[a] || *finValeur ||
                    valeur < 1 || valeur > INT_MAX) goto usage;
                if (strcmp(option, "-N") == 0)
                    nGrille = (int)valeur;
                else
                    k = (int)valeur;
            }
        } else {
            if (option[0] == '-' || fichier) goto usage;
            fichier = option;
        }
    }
    if (!fichier || nGrille < 3 ||
        nGrille > INT_MAX / nGrille) goto usage;

    FILE *entree = fopen(fichier, "r");
    if (!entree) {
        perror("Ouverture du nuage");
        return EXIT_FAILURE;
    }
    if (fscanf(entree, "%d", &N) != 1 || N <= k) {
        fprintf(stderr, "Nombre de points invalide.\n");
        fclose(entree);
        return EXIT_FAILURE;
    }

```

```

}
Grille grille = { .nx = nGrille, .ny = nGrille,
                  .xmin = 0.0, .ymin = 0.0, .h = 0.0,
                  .g = NULL, .f = NULL };
const size_t taille = (size_t)grille.nx * grille.ny;
int statut = EXIT_FAILURE;

/* 1. Allocation des points et du champ de sortie. */
double (*points)[2] = malloc((size_t)N * sizeof *points);
grille.f = calloc(taille, sizeof *grille.f);
if (!points || !grille.f) {
    fprintf(stderr, "Allocation impossible.\n");
    goto fin;
}

/* 2. Lecture des N points et de leur boite englobante. */
double xmin = INFINITY, xmax = -INFINITY;
double ymin = INFINITY, ymax = -INFINITY;
for (int i = 0; i < N; ++i) {
    if (fscanf(entree, "%lf %lf",
              &points[i][0], &points[i][1]) != 2 ||
        !isfinite(points[i][0]) || !isfinite(points[i][1])) {
        fprintf(stderr, "Point %d invalide ou manquant.\n", i);
        goto fin;
    }
    xmin = fmin(xmin, points[i][0]);
    xmax = fmax(xmax, points[i][0]);
    ymin = fmin(ymin, points[i][1]);
    ymax = fmax(ymax, points[i][1]);
}
fclose(entree);
entree = NULL;

/* Domaine fixe pour comparer plusieurs resolutions. */
double etendue = fmax(xmax - xmin, ymax - ymin);
if (!(etendue > 0.0) || !isfinite(etendue)) {
    fprintf(stderr, "Etendue du nuage invalide.\n");
    goto fin;
}
grille.h = 1.2 * etendue / (nGrille - 1);
grille.xmin = xmin - 0.1 * etendue;
grille.ymin = ymin - 0.1 * etendue;
if (sigma == 0.0) sigma = 1.5 * grille.h;

/* 3. Reconstruction complete : champ f et niveau. */
double niveau;
if (reconstructionPoisson(N, (const double (*)[2])points,
                          k, sigma,
                          grille.xmin, grille.ymin, grille.h,
                          grille.nx, grille.ny,
                          grille.f, &niveau) != 0) {
    fprintf(stderr, "Echec de la reconstruction.\n");
    goto fin;
}

/* 4. Export du champ et du niveau pour le trace. */
if (exporterResultats("reconstruction.dat", N,
                     (const double (*)[2])points,
                     &grille, niveau) != 0) {
    fprintf(stderr, "Echec de l'export.\n");
}

```

```
        goto fin;
    }
    statut = EXIT_SUCCESS;

fin:
    /* 5. Liberation, y compris apres une erreur. */
    if (entree) fclose(entree);
    free(grille.f);
    free(points);
    return statut;

usage:
    fprintf(stderr, "Usage : %s nuage.txt -N taille "
                  "[-sigma longueur] [-k voisins]\n", argv[0]);
    return EXIT_FAILURE;
}
```

La fonction `reconstructionPoisson` alloue et libère ses données internes : adjacences, normales, poids, divergence, matrices creuses et vecteurs de résolution. Elle conserve le tableau de sortie `f`, qui appartient à la fonction C appelante. Dans le `main`, le champ `g` de la structure `Grille` reste inutilisé ; l'export utilise uniquement la géométrie de la grille et `f`. Le fichier exporté sert à tracer la ligne $f_h = \tau$, superposée au nuage.

Références

- [1] M. Kazhdan, M. Bolitho et H. Hoppe, *Poisson Surface Reconstruction*, Symposium on Geometry Processing, 2006. Le présent devoir en propose une adaptation pédagogique en 2D sur grille cartésienne régulière.
- [2] M. Muja et D. G. Lowe, *Fast Approximate Nearest Neighbors with Automatic Algorithm Configuration*, International Conference on Computer Vision Theory and Applications (VISAPP), 2009, p. 331–340.
- [3] G. Karypis, *METIS: A Software Package for Partitioning Unstructured Graphs, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices*, version 5.1.0, University of Minnesota, 2013.